

그리디 알고리즘

권용환

2026 ALOHA 초급반 2주차

1. 그리디 알고리즘

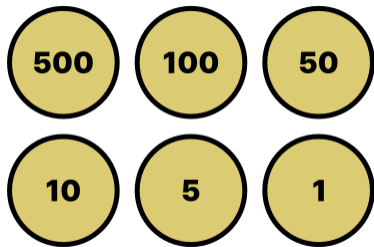


- ✓ 그리디 알고리즘이란 매 순간 최선의 선택을 쌓아 가며 결과적으로 최적인 답을 구해내는 알고리즘이다.
- ✓ 최선의 선택이란 무엇일까? 예시를 통해 알아보자.

1. 그리디 알고리즘



- ✓ 500원, 100원, 50원, 10원, 5원, 1원 동전이 있다.
- ✓ 이 동전들로 N 원을 만들되, 동전의 개수의 합을 최소로 하고 싶다.
- ✓ 어떻게 동전들을 선택해야 할까? 즉 각 동전의 개수를 몇 개씩으로 해야 할까?





- ✓ 750원을 만들어 보자.
- ✓ 먼저 500원을 1개 선택한다. 250원이 남는다.
- ✓ 이제 500원을 선택할 수 없으니 100원을 선택한다. 100원은 2개 선택할 수 있다. 50원이 남는다.
- ✓ 이제 100원을 선택할 수 없으니 50원을 선택한다. 50원은 1개 선택할 수 있다. 0원이 남는다.
- ✓ 선택한 동전의 총 개수는 500원 1개, 100원 2개, 50원 1개로 총 4개이다.



- ✓ 특정 금액을 만들 때 작은 금액의 동전을 사용하는 것보다 큰 금액의 동전을 사용하는 것이 동전 개수를 줄일 수 있다.
- ✓ 이와 같이 우리는 직관적으로 큰 금액의 동전부터 선택하는 것이 이득임을 알 수 있다.
- ✓ 매 순간 최선의 선택을 쌓았고, 결과적으로 최적의 답을 얻었다.

1. 그리디 알고리즘



- ✓ 그리디 알고리즘의 핵심은 이후의 상황을 고려하지 않고 항상 현재 상황에서 이득인 선택만을 반복해도 최선의 결과를 얻을 수 있는 것이다.
- ✓ 하지만 모든 문제에 그리디 알고리즘을 적용할 수는 없다.

1. 그리디 알고리즘

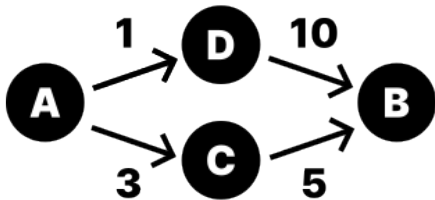


- ✓ 동전의 금액이 500원, 200원, 10원으로 바뀌었다고 생각해 보자.
- ✓ 600원을 가장 적은 동전 개수로 만들어 보자.
- ✓ 200원을 3개 사용하는 것이 가장 동전 개수가 적다는 것을 직관적으로 알 수 있다.
- ✓ 하지만 그리디 알고리즘을 이용한다면 500원짜리를 1개 선택하고 10원짜리를 10개 선택하게 된다.
- ✓ 이처럼 그리디 알고리즘을 사용하면 안 되는 때도 있다.

1. 그리디 알고리즘



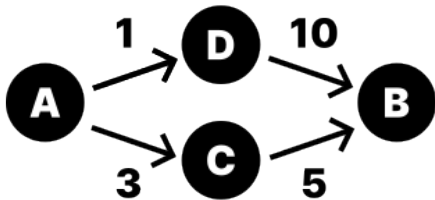
- ✓ 그리디 알고리즘을 사용하면 안 되는 예시를 하나만 더 알아보자.
- ✓ 다음과 같은 경로가 있다. 각 경로에 적힌 수는 거리를 의미한다.
- ✓ A에서 B까지 최단 경로, 즉 거리의 합이 가장 적게 이동하려면 어떻게 이동해야 할까?



1. 그리디 알고리즘



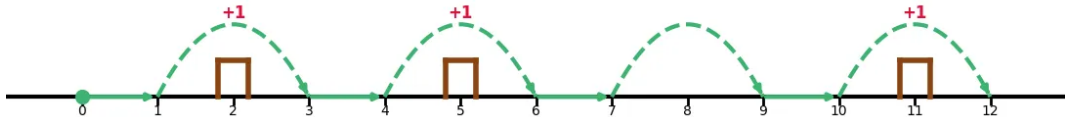
- ✓ 그리디 알고리즘은 항상 현재 상황에서 가장 이득인 결정만 한다.
- ✓ A에서는 C 또는 D로 이동할 수 있다. D로 이동하는 것이 C로 이동하는 것보다 거리가 작으므로 D로 이동한다.
- ✓ 따라서 그리디 알고리즘이 구한 최단 경로는 $A \rightarrow D \rightarrow B$ 가 된다.
- ✓ 하지만 실제 최단 경로는 $A \rightarrow C \rightarrow B$ 인 것을 쉽게 알 수 있다.



1. 그리디 알고리즘



- ✓ 그리디 알고리즘을 활용하는 연습 문제인 '장애물'(JUNGOL 8604)을 풀어 보자.
- ✓ 위치 0에서부터 시작해 수직선 위에 놓인 N 개의 장애물을 모두 뛰어 넘어야 한다.
- ✓ 걸기를 하면 위치 x 에서 $x + 1$ 로, 뛰기를 하면 x 에서 $x + 2$ 로 이동한다.



- ✓ 걸기와 뛰기 횟수의 합인 이동 횟수를 최소화해 보자.



- ✓ 장애물이 하나도 없다고 생각해 보자. 이동 횟수를 최소화하기 위해선 뛰는 횟수를 최대화하는 것이 이득이다.
- ✓ 장애물이 하나도 없고, 거리 d 를 이동해야 한다면 이동 횟수의 최솟값은 $\lceil d/2 \rceil$ 이다. 이때 $\lceil \rceil$ 은 올림 기호이다.
- ✓ 이제 문제로 돌아오자. 장애물 N 개에 대해서는 항상 뛰어야 한다. 따라서 이동 횟수에 N 이 추가된다.
- ✓ 모든 장애물을 넘어가는 것이 불가능한 경우가 있을까?

1. 그리디 알고리즘



- ✓ 어떤 두 장애물의 위치를 x, y ($x < y$)라 할 때 $y - x = 1$ 인 경우가 존재한다면 불가능하다.
- ✓ 왜냐하면 첫 장애물을 넘기 위해 위치 $x - 1$ 에서 뛰다면 위치 $x + 1$, 즉 위치 y 에 도착하기 때문이다.



- ✓ 이제 각 장애물 사이 거리를 이동하기 위한 이동 횟수의 최솟값을 구해 보자.
- ✓ i 번째 장애물을 넘고 난 위치는 $X_i + 1$ 이다. $i + 1$ 번째 장애물을 넘으려면 위치 $X_{i+1} - 1$ 까지 이동해야 하므로 거리 $X_{i+1} - X_i - 2$ 를 이동하기 위한 이동 횟수의 최솟값을 구하면 된다.
- ✓ 또한 첫 번째 장애물을 넘기 전까지 이동해야 하는 거리인 $X_1 - 1$ 을 이동하기 위한 이동 횟수의 최솟값도 구해 더해 주자.
- ✓ 그러면 총 이동 횟수의 최솟값은

$$N + \left\lceil \frac{X_1 - 1}{2} \right\rceil + \sum_{i=1}^{N-1} \left\lceil \frac{X_{i+1} - X_i - 2}{2} \right\rceil$$

이다.

1. 그리디 알고리즘



- ✓ C++로 그대로 구현하면 된다.
- ✓ 올림 $\lceil a/b \rceil$ 는 두 가지 방법으로 구현할 수 있다.

```
int f(int a, int b) {  
    if (a % b == 0) return a / b;  
    else return a / b + 1;  
}
```

```
(a + b - 1) / b
```

1. 그리디 알고리즘



✓ 아래는 예제 코드이다.

```
#include <iostream>
using namespace std;
int x[250'005];

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, ans = 0; cin >> n;
    for (int i = 0; i < n; i++) cin >> x[i];
    for (int i = 0; i + 1 < n; i++) {
        if (x[i + 1] - x[i] == 1) {
            cout << "-1\n";
            return 0;
        }
    }
    for (int i = 1; i < n; i++) {
        ans += (x[i] - x[i - 1] - 2 + 1) / 2;
    }
    cout << ans + n + x[0] / 2 << "\n";
    return 0;
}
```

2. 연습 문제



- ✓ **JUNGOL 8604** 장애물
앞에서 해설한 문제이다.
- ✓ **JUNGOL 8962** 사과
사과 사는 문제.
- ✓ **JUNGOL 21048** 금메달은 사실 은메달이야!!
금메달과 은메달이 인접한 경우를 고려해보자.
- ✓ **JUNGOL 5125** 피하자
짜수를 왼쪽으로 몰아넣기 vs 오른쪽으로 몰아넣기

2. 연습 문제



- ✓ **JUNGOL 5749** 스케이트 연습
뒤에서부터...?
- ✓ **JUNGOL 1370** 회의실 배정
회의를 어떤 기준으로 선택해야 가장 많은 회의를 선택할 수 있을까?
- ✓ **JUNGOL 1828** 냉장고
최대 보관 온도를 기준으로 정렬해 보면 어떨까?
- ✓ **JUNGOL 4520** 햄버거 분배
먹을 수 있는 가장 가까운 햄버거 vs 먹을 수 있는 가장 왼쪽 햄버거